

Keddeh Systems — Substrate bootstrap staging and qualification

Document: KEX-RD-SUBSTRATE-20261010-001. Version 0.1. Status: engineering draft, local qualification only. Classification: owner engineering review. Architecture attribution: Aboudy Keddeh. Implementation and analysis contribution: Codex. Publisher and rights holder: Keddeh Systems / Aboudy Keddeh. Report rights reserved; existing source licences remain unchanged. Exact publishing-template assets are unavailable; this is a draft with explicit metadata, not certified template conformity.

1. Requirement and source custody

Preserve working services; stage supplied source before deployment. Require secured runtime paths, canonical atomic state, bounded shared-memory views, watchdog recovery, UDS bidirectional handshake and API startup withholding. Local unity never authorizes merging independent governed contexts. GitHub revision 09f45187dbec29ebe63153f916b67505452dc136 was fetched and archived without changing the active KEDDEH-- checkout. STAGING_REVIEW.json records the archive and per-file digests. Notebook-export ServerSpace sources were inspected as data; their embedded shell and administrative commands were not run.

2. Baseline assessment and integration decision

The supplied revision provides SDK service adapters and exit-based supervision. Inspection found no heartbeat recovery, shared-memory substrate, canonical state writer or UDS startup handshake in its estate supervisor. The restored machine has no /workspace/deployments/service-estate/qualification.json and retains 2d95818. The reported 15 SDK and 77 installer tests are prior supplied claims, not reproduced here. The operating ServerSpace host, secured runtime directory and baseline process configuration have not been established. Production replacement is withheld; the canary is independently addressable.

3. Implementation and identity boundaries

scripts/engineering/substrate_canary.py reuses context_continuation.context, digest and canonical. Context keys include origin, ordinance, municipality, region, vector, frame, unit and revision. The module does not interpret equal numeric units as equal authority. Origin, identity and carrier offsets remain distinct. Payloads are qualification state, not owner compiler instructions or IL-LLM semantic authority. The canonical file is a persistent backing carrier read through bounded mmap shared views. This is not a claim of a complete writable shared-memory allocator or 100 TB physical RAM provisioning.

4. State, transport and readiness contract

Owner-only 0700 runtime directories and 0600 leaf files/socket are required. Symlink traversal, unsafe file ownership, modes and hardlinks are rejected. flock serializes writers; temporary writes are fsynced and atomically renamed, followed by directory fsync. SHA-256 and CRC32 verify the canonical body. UDS frames use bounded length prefixes and exact reads. Same-UID peer credentials and HMAC-SHA256 nonce challenges complete in both directions. Readiness 7 comprises verified state, authenticated local peer and completed acknowledgement. Checksums alone are not peer authentication. These controls supplement the fixture transport; they do not replace native KEX execution lineage or constitute production peer enrollment.

5. Qualification method and observed results

13 unittest checks passed. Faults include killed and SIGSTOP-hung owned workers, corrupt state, wrong secret, oversized/truncated frames, concurrent writers and conflicting substrate ownership. Distinct parent contexts at local unit 1 remain separate. State is identical after watchdog recovery. 100 fresh authenticated exchanges completed with matching digest and mask 7; fragmented frames were reassembled. Canary workers enforce 256 MiB address space, 64 descriptors and disabled core dumps. These are fixture limits, not the production resource plan. API wrapper tests prove a marker command remains unexecuted until handshake; the real Mining API has not been deployed or tested.

6. Running canary and existing-estate restoration

python3 -B scripts/engineering/start_substrate_canary.py reconciles /workspace/.keddeh-environment/substrate-canary and verifies the authenticated worker. Repeat startup retained the same worker and state digest. Only a qualification context is initialized when state is absent; corrupt existing state prevents startup. Existing registry, circuits, VFS, Site review, queue and WEB4 startup checks completed through the held estate helper. Queue restoration initially rejected stale process metadata; inspection confirmed its managed process was gone. The helper now retains that state and selects a fresh private managed directory. No unknown process was signalled.

7. Critique, uncertainty and emergent learning

Process-exit supervision alone cannot detect a live hung process; active authenticated heartbeat testing exposed the missing requirement and the canary tests both modes. Atomic rename preserves committed state across process restarts but an already-open mmap view retains its old inode; consumers reopen views after the handshake. This is a deliberate bounded snapshot contract, not in-place live write sharing. Local key possession identifies an admitted same-UID fixture peer, not a remote certificate hierarchy. A same-UID hostile process is outside this isolation boundary. Tested IPC acknowledgement does not prove loss-free arbitrary networks, power-loss durability, HA across hosts, or universal cache correctness. No profound performance or energy result is asserted from these tests.

8. Deployment gates and next task

Hold broad rollout until the operating ServerSpace connection, secured directory, enrolled peer identity, readiness-bit specification, cache inventory and actual Mining API launch contract are retrieved. Compare the staged adapter with that baseline, add compatibility tests, run an isolated target-node canary, then qualify real API withholding, cache CRCs, frame acknowledgements, routing and public/internal endpoints. Promote incrementally only with baseline and rollback receipts. The generic gate command accepts an explicit reviewed argv after authentication; it is not automatically attached to an unknown API. New environment publication remains a product operation and is separate from draft persistence and local service readiness.